

StormWatch System Overview and Architecture

Document ID: SW-001 Category: system-overview Version: 1.0 Status: Synthetic workshop reference Security classification: workshop-public

Purpose

StormWatch is a teaching application that demonstrates how a .NET application, a local Model Context Protocol server, a hosted language model, and a retrieval index can work together without blurring their responsibilities.

The application resolves a city, reads OpenWeather's five-day forecast in three-hour periods, and calculates a transparent educational storm-risk signal. It can present the result through a command line, two local MCP tools, a Foundry-backed validation client, and an optional server-rendered web page.

StormWatch is not a meteorological forecast, emergency alert, medical device, travel clearance system, or safety system. Its score is a deterministic software exercise. Operational decisions must use official local weather and emergency services.

User Stories

- A learner asks for the next five forecast periods for a city.
 - A learner asks for an explainable educational risk assessment.
 - A learner asks how a score or threshold works and receives a cited answer.
 - A developer asks how the local MCP server and hosted model communicate.
 - A reviewer verifies that credentials and dynamic data remain on the correct side of each trust boundary.
- A chatbot recognizes when it lacks current tool data and does not substitute an example from the knowledge corpus.

Core Responsibilities

StormWatch separates four application responsibilities:

1. The weather boundary geocodes the city, requests forecast data, validates responses, and maps JSON into immutable domain records.
2. The risk boundary scores typed forecast points using deterministic rules. It performs no HTTP requests and knows nothing about MCP or user interfaces.
3. Presentation adapters render or return the typed result. The CLI, MCP tools, and web page must not copy weather parsing or risk rules.
4. The model orchestration boundary decides whether to retrieve documents, call an MCP tool, or combine both sources in one answer.

This separation makes tests deterministic and prevents rule drift between user interfaces.

Runtime Components

Component	Runs where	Responsibility
StormWatch application	Participant laptop	Weather parsing, risk scoring, typed results
StormWatch MCP server	Participant laptop	Advertises and executes two structured tools over stdio
GitHub Copilot	Hosted service plus editor client	Development assistance and optional MCP testing
Foundry model deployment	Microsoft Foundry	Chooses tools and produces a grounded conversational answer
Local Foundry client	Participant laptop	Starts MCP server, discovers tools, sends model requests, executes tool calls
OpenWeather	External HTTPS service	Geocoding and five-day, three-hour forecast data
Search index	Workshop-selected search service	Stores chunks and embeddings from the static PDF corpus
Optional web frontend	Participant laptop	Server-rendered presentation over existing services

Dynamic MCP Flow

For a city-specific question, the local Foundry client starts the StormWatch MCP server as a child process. It discovers exactly two tools: `get_forecast` and `assess_storm_risk`. Tool descriptions are supplied to the hosted model.

When the model requests a tool call, the local client executes it through local stdio. The MCP server obtains either an approved live forecast or the configured synthetic fixture, runs the existing application services, and returns concise structured data. The local client sends the selected tool result back to the model, which writes the final answer.

Azure does not connect inbound to the participant laptop. Model requests travel outbound over HTTPS. MCP traffic between the local client and StormWatch stays on the laptop through stdio.

Static RAG Flow

The RAG corpus contains stable knowledge: scoring policy, field definitions, architecture, operating guidance, examples, and limitations. A typical RAG request follows these steps:

1. Convert the user's question to a search query or embedding.
2. Retrieve a small set of relevant chunks from the PDF index.
3. Preserve each chunk's document ID, filename, section, and page number.
4. Give the chunks to the model as evidence, not as executable instructions.
5. Require the answer to cite the source PDF and page.
6. If the question needs current city facts, call MCP before answering.

RAG must not turn a training scenario into a current forecast. The scenarios are fixed examples designed to exercise retrieval and scoring explanations.

Source Precedence

Use the following source order when sources overlap:

1. A successful MCP result is authoritative for the requested city's current or configured fixture forecast and assessment.
2. The retrieved corpus is authoritative for stable StormWatch rules, tool contracts, architecture, and responsible-use guidance.
3. Scenario documents are authoritative only for the named training scenario.

4. General model knowledge may improve wording but may not invent StormWatch thresholds, tool output, weather facts, or safety claims.

If an MCP result and a scenario contain different values, the chatbot must state that the scenario is training data and use the MCP values for the city request.

Trust Boundaries

Credentials

OpenWeather and Foundry credentials enter only through the local process environment or an ignored facilitator-supplied .env file. They must never appear in source, committed configuration, chat prompts, commands, screenshots, logs, exceptions, URLs returned to users, PDF content, MCP results, or browser responses.

Model-visible data

Conversation content and selected tool results are sent to the Foundry model. The workshop therefore uses only synthetic fixture and corpus data. Customer, patient, employee, operational, or other restricted information must not be substituted.

Browser boundary

The optional browser calls only the local Razor Pages server. The server calls the existing application services. The browser must not call OpenWeather directly and must never receive an API key or a URL containing appid=.

MCP stdout boundary

In stdio mode, standard output carries MCP protocol messages. Application reports and normal host logging must not be written there because they could corrupt the protocol stream.

Grounded Answer Pattern

A well-grounded answer distinguishes facts by source:

- Dynamic fact: "The configured StormWatch assessment reports WATCH (33/100) at 12:00 UTC." Cite the MCP tool result.
- Stable explanation: "A score from 30 through 59 maps to WATCH." Cite the risk scoring PDF.
- Qualification: "This is an educational heuristic, not an official warning."
- Escalation: "Check official local weather and emergency services for operational decisions."

The model must say that data is unavailable when retrieval or a required tool fails. It must not fill gaps with a city scenario, remembered weather, or a plausible-looking score.